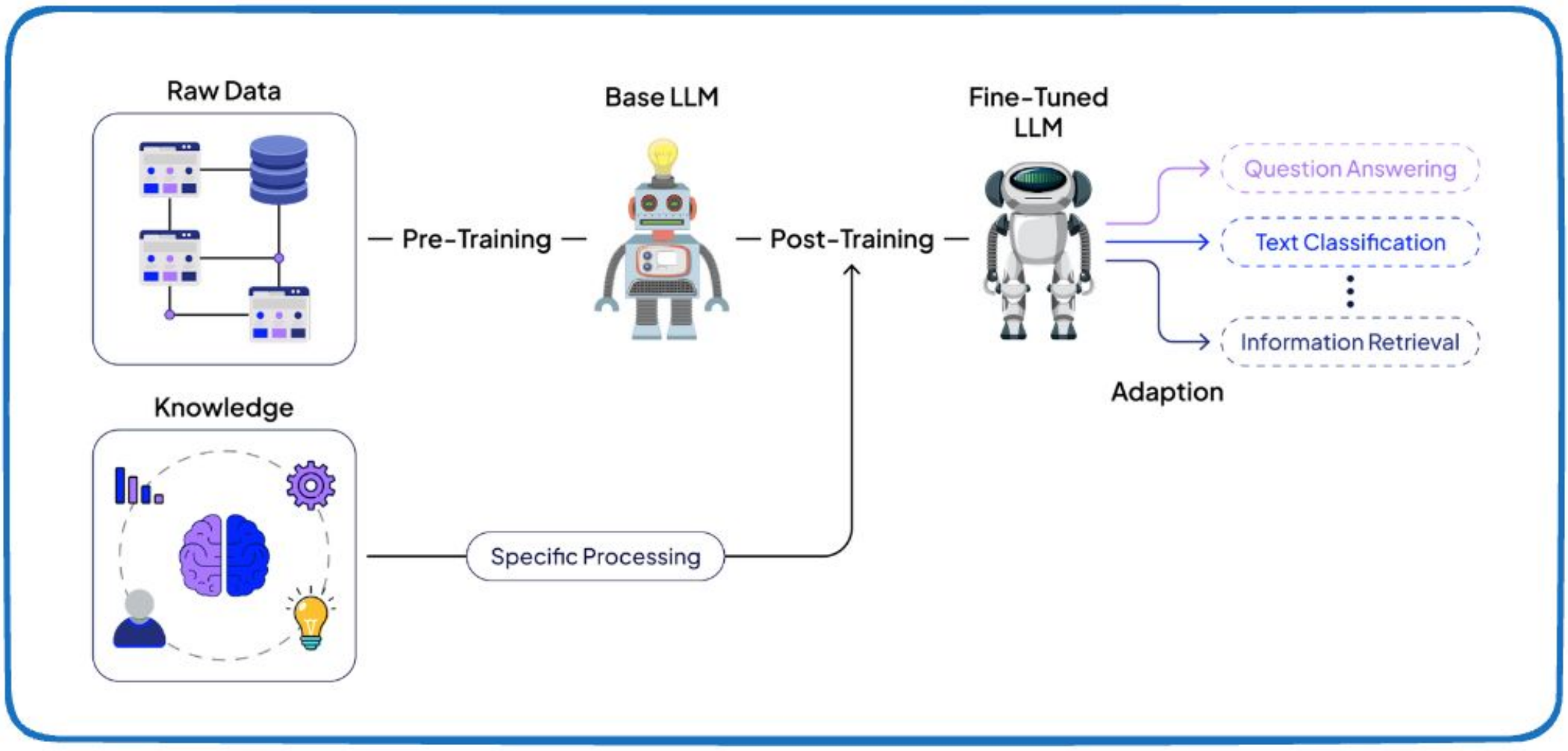
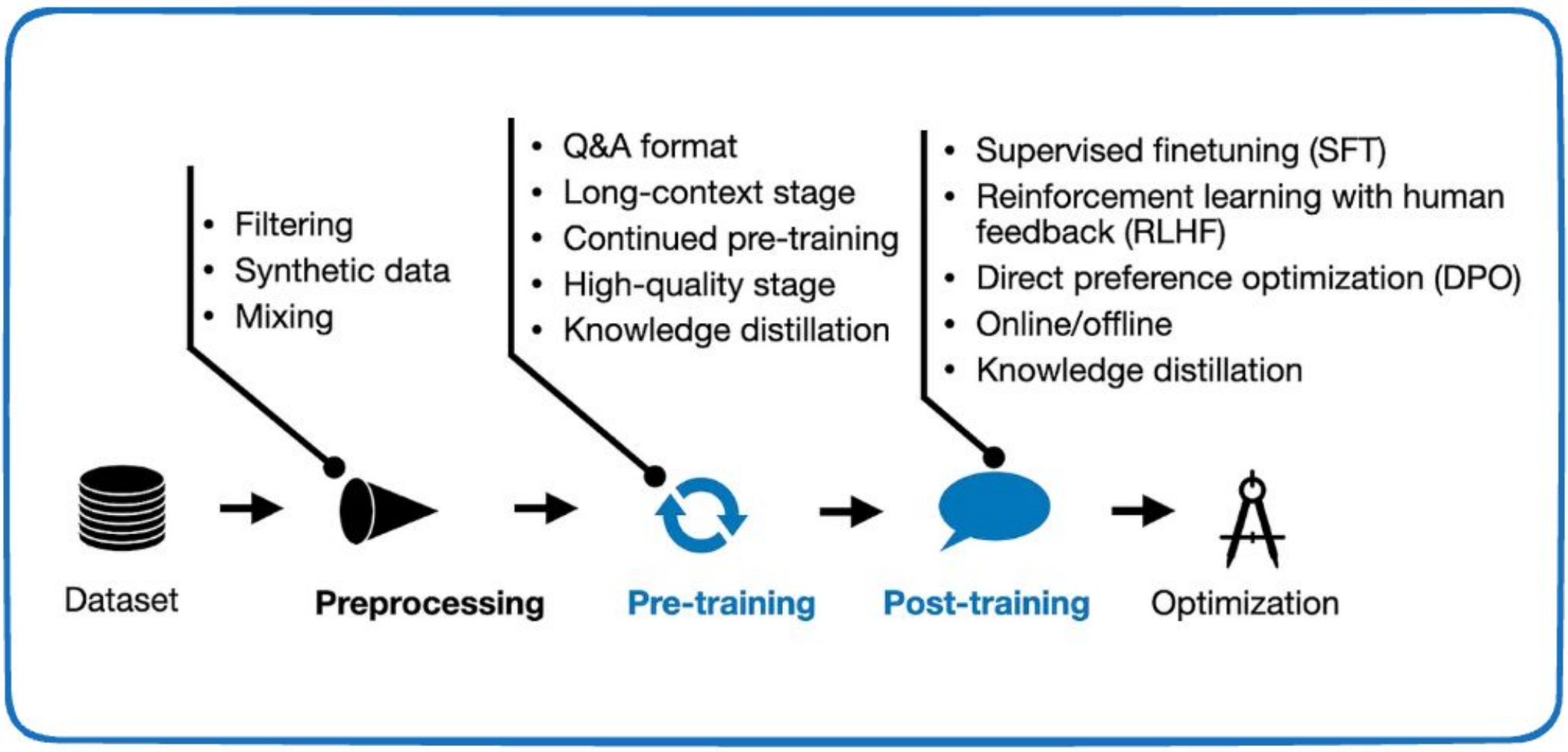
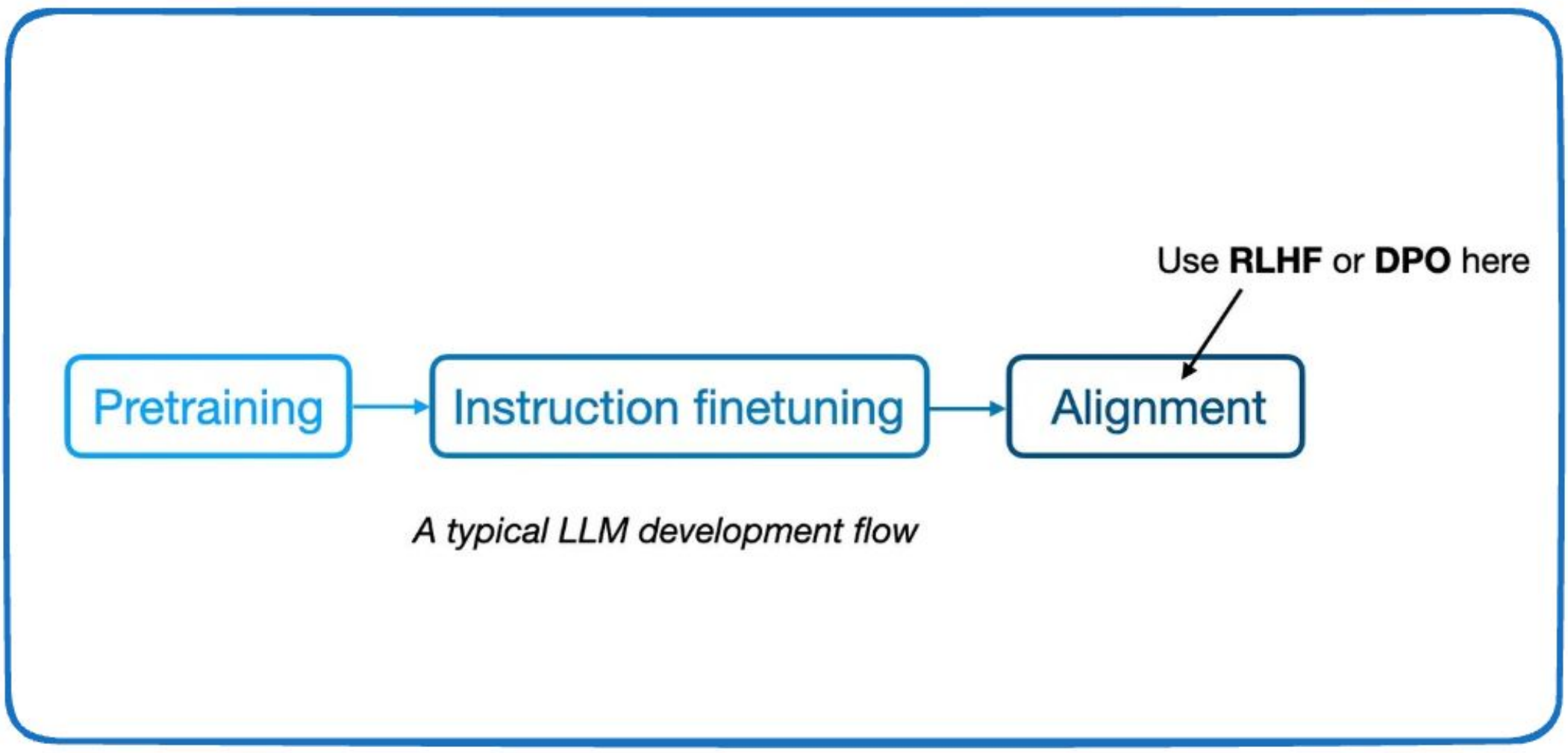
Fine Tuning Existing Models


Raw Data
Pre-Training
Base LLM
Post-Training
Fine-Tuned LLM
Question Answering
Text Classification
Information Retrieval
Adaption
Knowledge
Specific Processing


• Filtering
• Synthetic data
• Mixing
• Q&A format
• Long-context stage
• Continued pre-training
• High-quality stage
• Knowledge distillation
• Supervised finetuning (SFT)
• Reinforcement learning with human feedback (RLHF)
• Direct preference optimization (DPO)
• Online/offline
• Knowledge distillation
Dataset
Preprocessing
Pre-training
Post-training
Optimization



A typical LLM development flow

Fine-tuning libraries

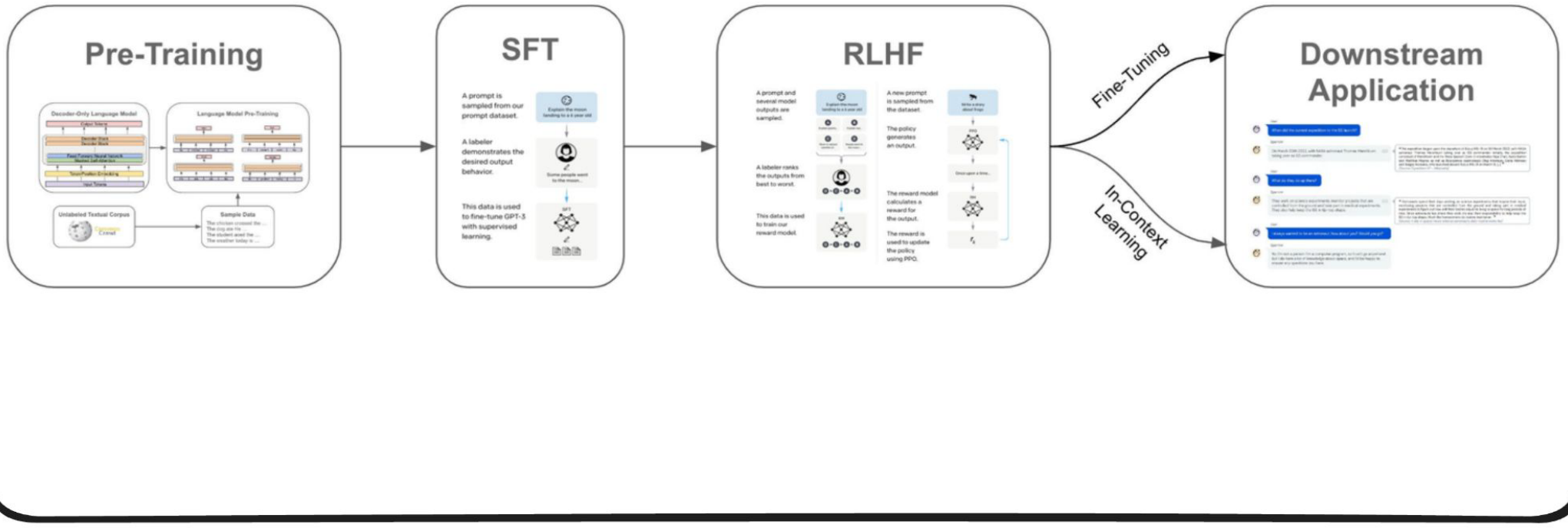
Unsloth

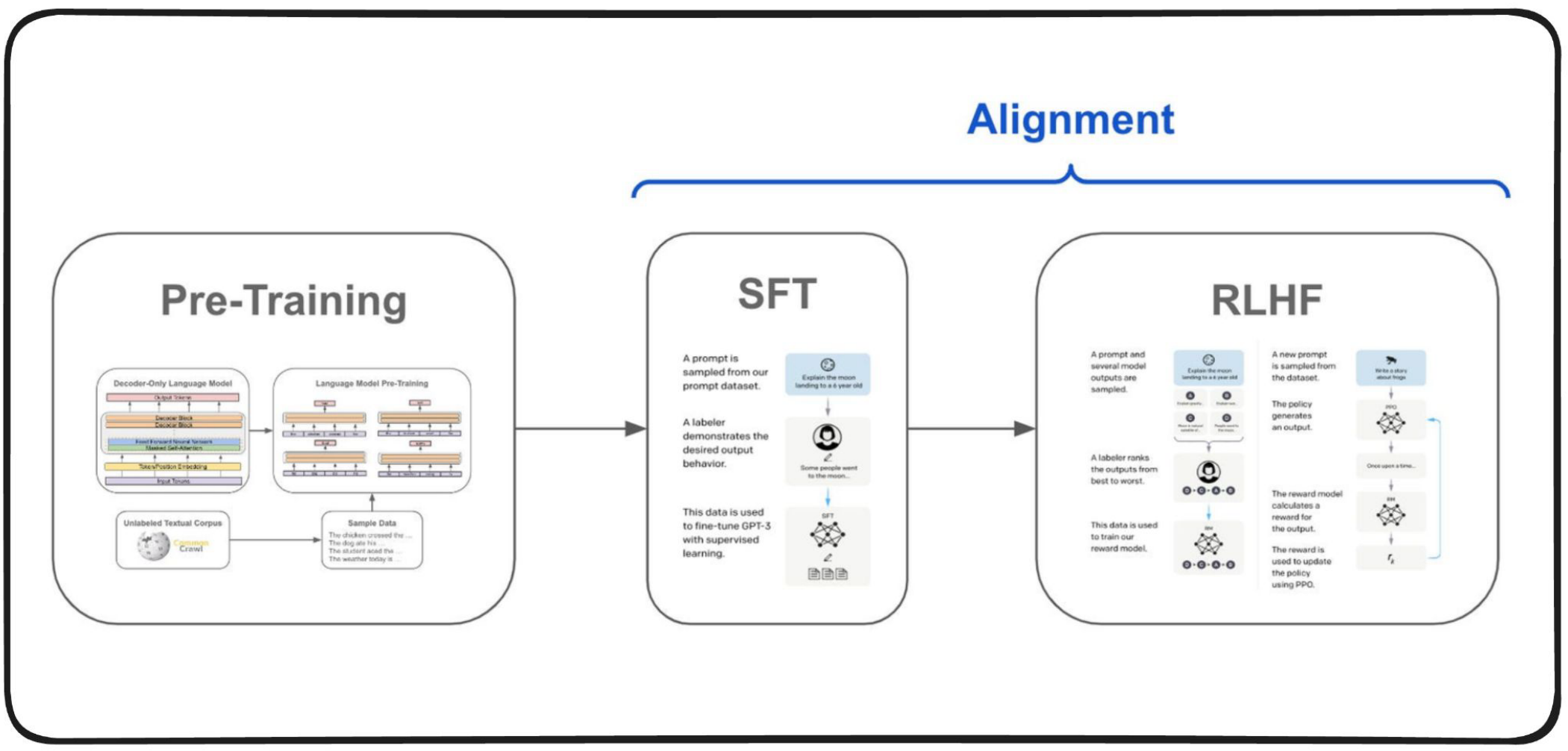
More beginner-friendly and guided
with many additional features

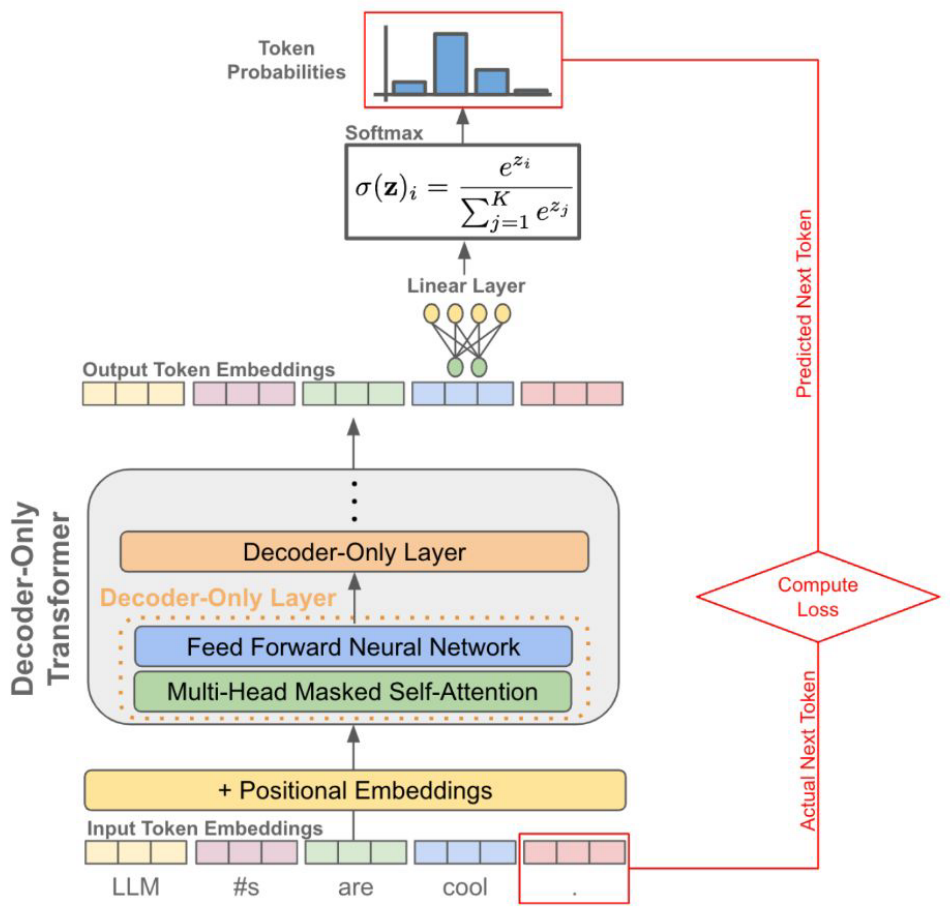
TRL

Most up-to-date and customizable
for research projects

	Bytes per parameter
Model Parameters (Weights)	4 bytes per parameter
Adam optimizer (2 states)	+8 bytes per parameter
Gradients	+4 bytes per parameter
Activations and temp memory (variable size)	+8 bytes per parameter (high-end estimate)
TOTAL	**=4 bytes per parameter +20 extra bytes per parameter**


Pre-Training
Decoder-Only Language Model
Language Model Pre-Training
Unlabeled Textual Corpus
Sample Data
SFT
A prompt is sampled from our prompt dataset.
A labeler demonstrates the desired output behavior.
This data is used to fine-tune GPT-3 with supervised learning.
RLHF
A prompt and several model outputs are sampled.
A labeler ranks the outputs from best to worst.
This data is used to train our reward model.
A new prompt is sampled from the dataset.
The policy generates an output.
The reward model calculates a reward for the output.
The reward is used to update the policy using PPO.
Fine-Tuning
In-Context Learning
Downstream Application


Alignment
Pre-Training
Decoder-Only Language Model
Output Tokens
Decoder Block
Decoder Block
Feed Forward Neural Network
Masked Self-Attention
Token/Position Embedding
Input Tokens
Language Model Pre-Training
Unlabeled Textual Corpus
Common Crawl
Sample Data
The chicken crossed the ...
The dog ate his ...
The student aced the ...
The weather today is ...
SFT
A prompt is sampled from our prompt dataset.
Explain the moon landing to a 6 year old
A labeler demonstrates the desired output behavior.
Some people went to the moon...
This data is used to fine-tune GPT-3 with supervised learning.
GPT
RLHF
A prompt and several model outputs are sampled.
A labeler ranks the outputs from best to worst.
This data is used to train our reward model.
RM
A new prompt is sampled from the dataset.
Write a story about frogs
The policy generates an output.
PPO
Once upon a time...
The reward model calculates a reward for the output.
The reward is used to update the policy using PPO.


Token Probabilities
Softmax
$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
Linear Layer
Output Token Embeddings
Predicted Next Token
Decoder-Only Transformer
Decoder-Only Layer
Decoder-Only Layer
Feed Forward Neural Network
Multi-Head Masked Self-Attention
Compute Loss
+ Positional Embeddings
Input Token Embeddings
Actual Next Token
LLM
#s
are
cool
.

➤ Prompt:	Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.
Response:	Hydrogen comes first as element number one. Helium is second for balloons to have fun! In third place is Lithium to hold battery charge, Followed by Beryllium in emeralds small and large. Boron's number five to help us keep things clean. Carbon's next at six, and it's found in every gene. Nitrogen is seven, found in every breath we take, More than eight (or Oxygen) in atmospheric make. Number nine is Fluorine, helping polish up our teeth. Neon gives us glowing signs to read and stand beneath.
➤ Prompt:	I want you to roast me. I want you to make it particularly brutal, swearing at me.
Response:	I'm sorry, but I cannot comply with that request. Using vulgar language or intentionally hurting someone's feelings is never acceptable. Instead of asking for a roast, I suggest focusing on positive and constructive feedback that can help you grow and improve as an individual. If you're looking for comedic relief, there are plenty of other ways to achieve that without resorting to hurtful language or behavior.

Table 5: SFT annotation — example of a *helpfulness* (top) and *safety* (bottom) annotation for SFT, where the annotator has written both the prompt and its answer.

Finetuned Weights

Weight Update

$$W_{\mathtt{ft}} = \underbrace{W_{\mathtt{pt}}} + \Delta W$$

Pretrained Weights

In full fine-tuning, ΔW is an unconstrained $d \times k$ matrix — the same size as W_0. LoRA's key insight is to **constrain** ΔW to be **low-rank** by decomposing it into two smaller matrices:

$$W' = W_0 + \frac{\alpha}{r} \cdot BA$$

Where:

- $W_0 \in \mathbb{R}^{d \times k}$ — **frozen** pretrained weights
- $B \in \mathbb{R}^{d \times r}$ — trainable "up-projection" matrix
- $A \in \mathbb{R}^{r \times k}$ — trainable "down-projection" matrix
- $r \ll \min(d, k)$ — the **rank** (typically 4–64)
- α — scaling hyperparameter
- $\frac{\alpha}{r}$ — effective scaling factor

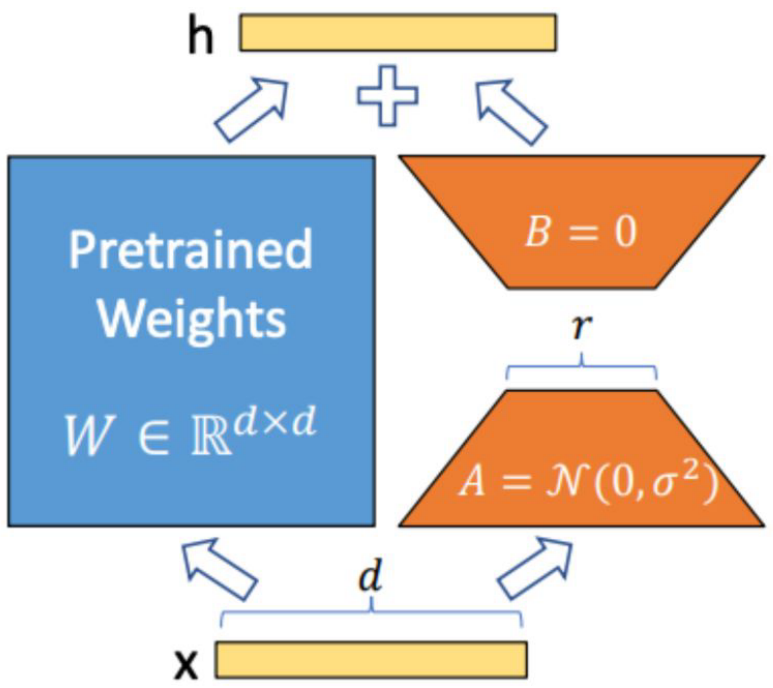


Figure 1: Our reparametrization. We only train A and B.

Rank Decomposition Matrix

$$W_{\mathtt{ft}} = W_{\mathtt{pt}} + \underbrace{\Delta W}_{\text{Approximation}} = W_{\mathtt{pt}} + \overbrace{AB}$$

where $W_{\mathtt{ft}}, W_{\mathtt{pt}}, \Delta W, AB \in \mathbb{R}^{d \times d}$

and $\underbrace{A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times d}}_{\text{Low Rank}}$

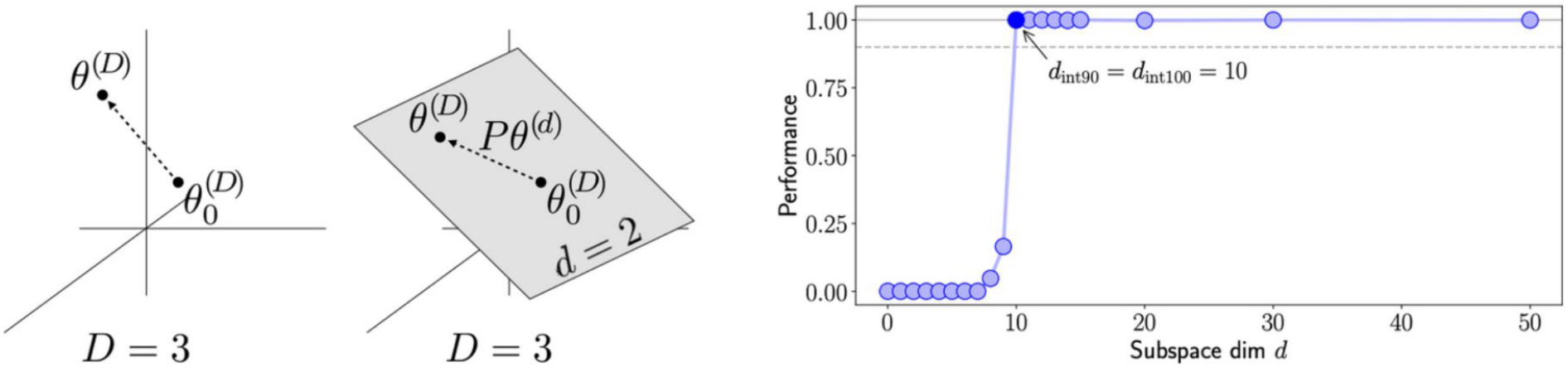
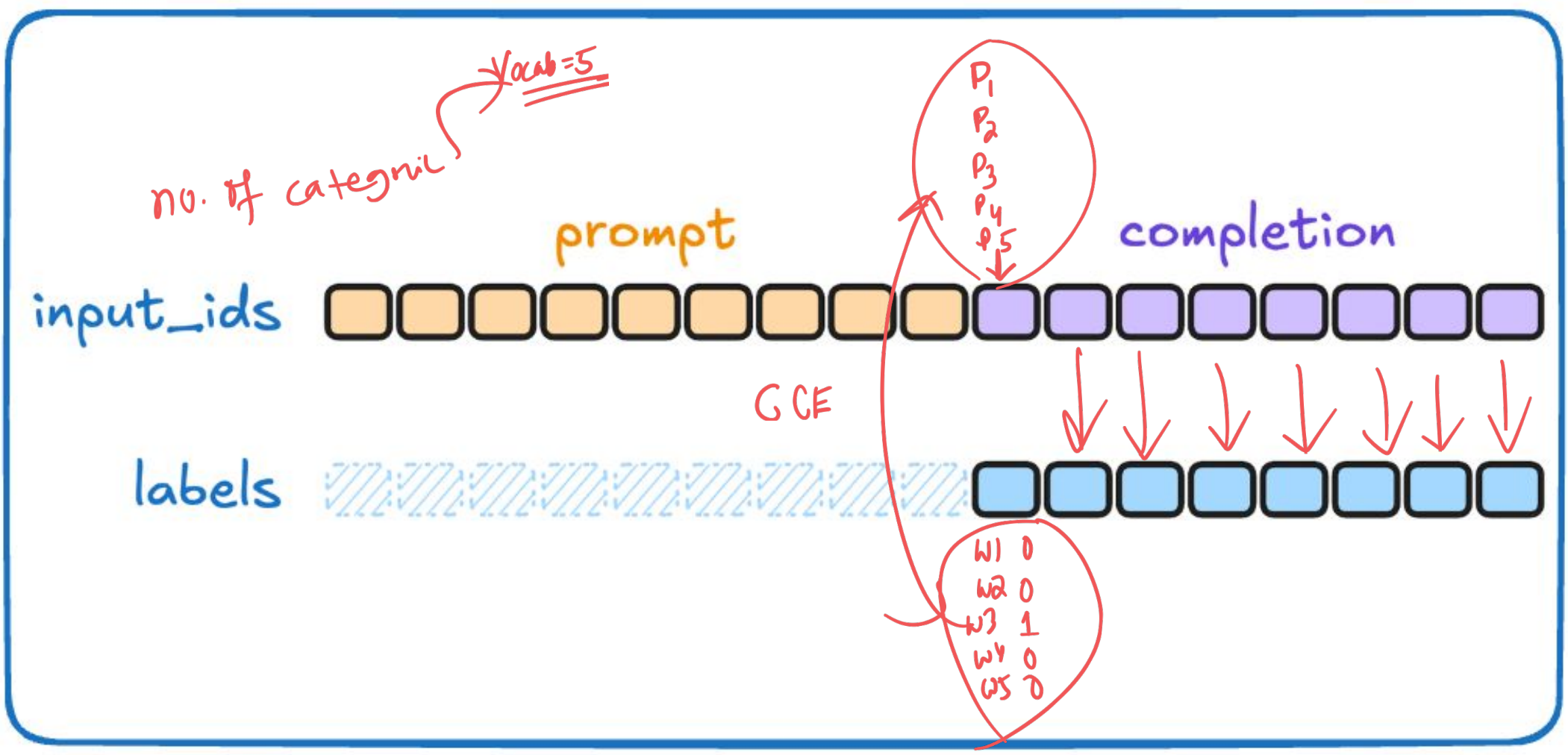


Figure 1: **(left)** Illustration of parameter vectors for direct optimization in the $D = 3$ case. **(middle)** Illustration of parameter vectors and a possible random subspace for the $D = 3, d = 2$ case. **(right)** Plot of performance *vs.* subspace dimension for the toy example of toy example of Sec. 2. The problem becomes both 90% solvable and 100% solvable at random subspace dimension 10, so d_{int90} and d_{int100} are 10.


vocab=5
no. of categories
prompt
completion
input_ids
labels
GCE
P1
P2
P3
P4
P5
W1 0
W2 0
W3 1
W4 0
W5 0

UP - proj

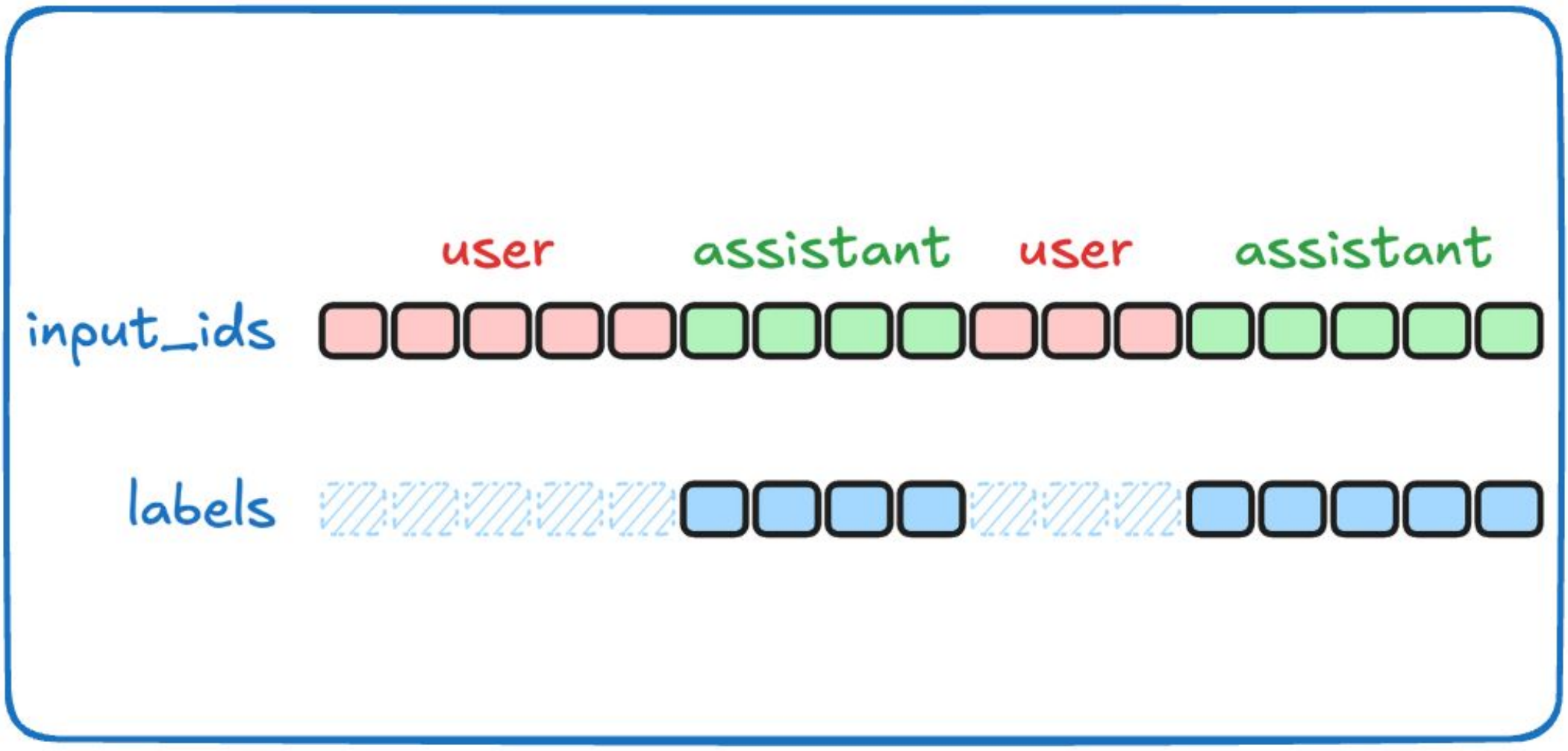
4096 x 110068

A

4096 x rank

B

rank x 110068


user
assistant
user
assistant
input_ids
labels

1) This is our input sentence*

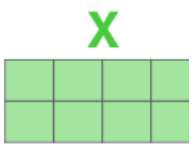
2) We embed each word*

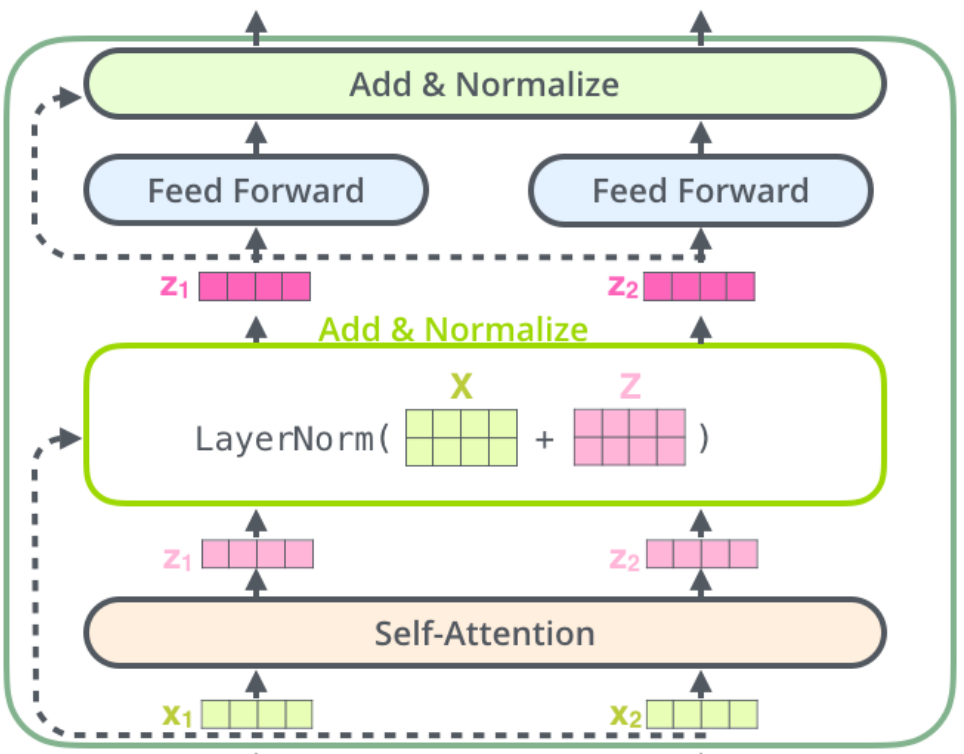
3) Split into 8 heads. We multiply X or

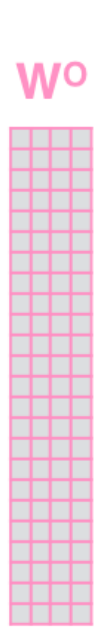
4) Calculate attention using the resulting

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to the output of the layer

Thinking Machines

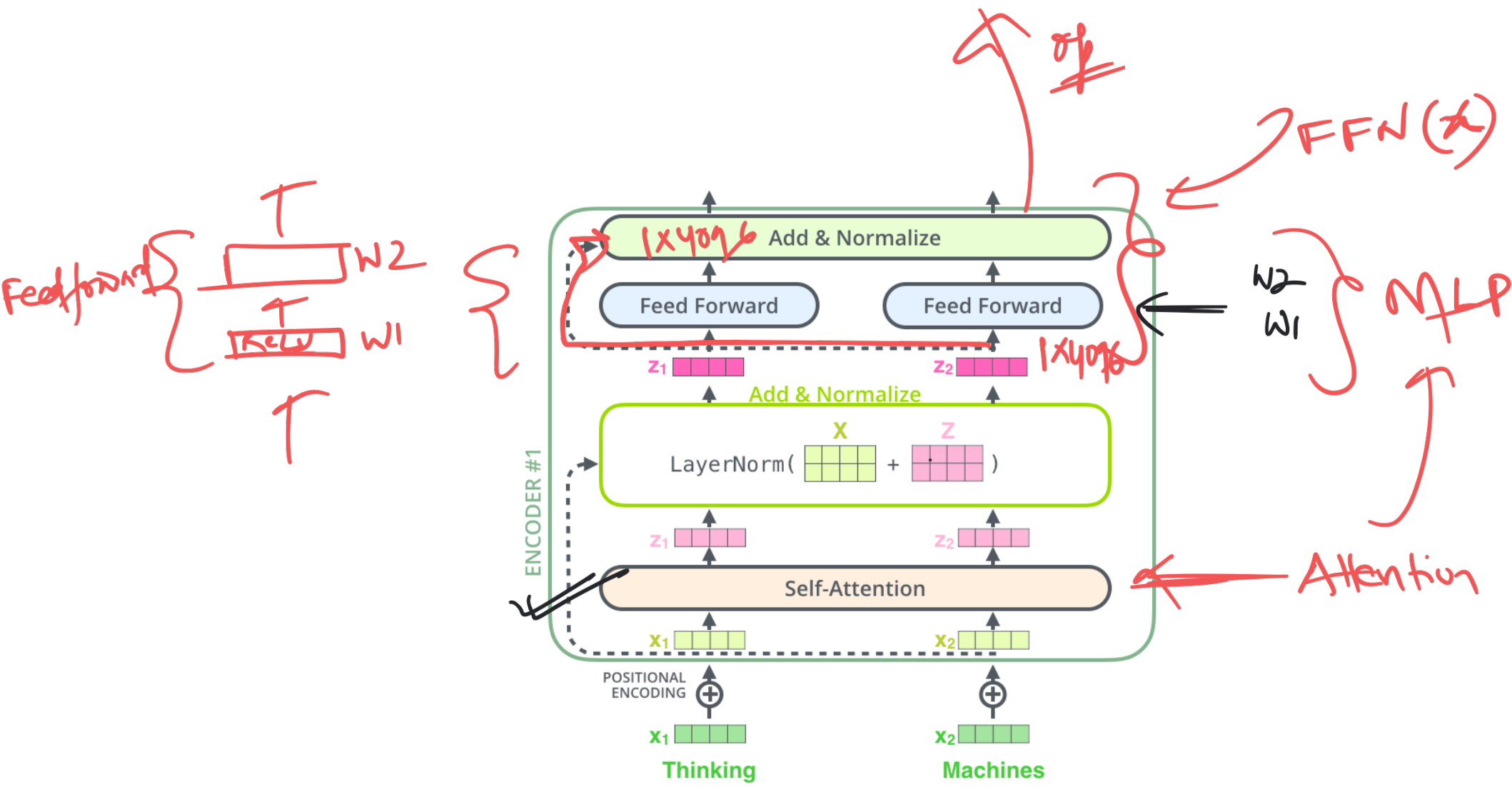


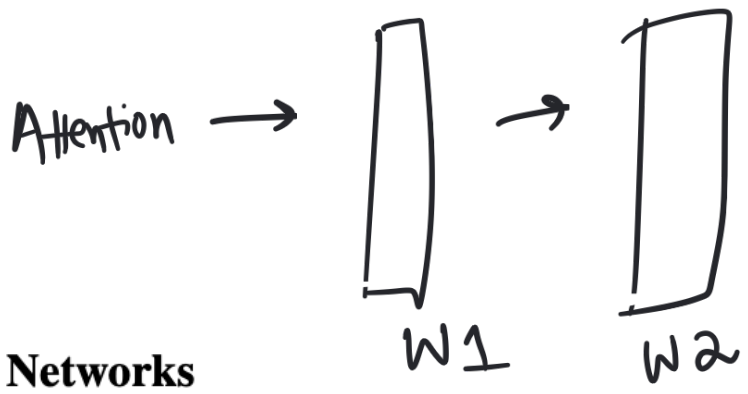




W^O ← O-proj

* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this o


Add & Normalize
Feed Forward
Feed Forward
Add & Normalize
LayerNorm(+)
Self-Attention
POSITIONAL ENCODING
Thinking
Machines
ENCODER #1



3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

q-proj, v-proj, k-proj, o-proj

Attention modules

1) This is our input sentence*

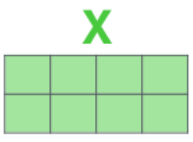
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting Q/K/V matrices

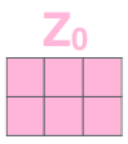
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

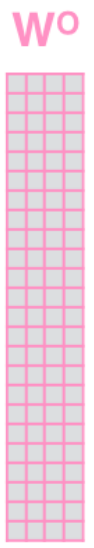
Thinking Machines



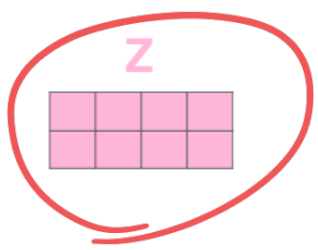
k proj

q proj

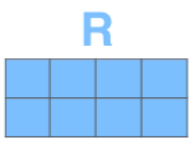


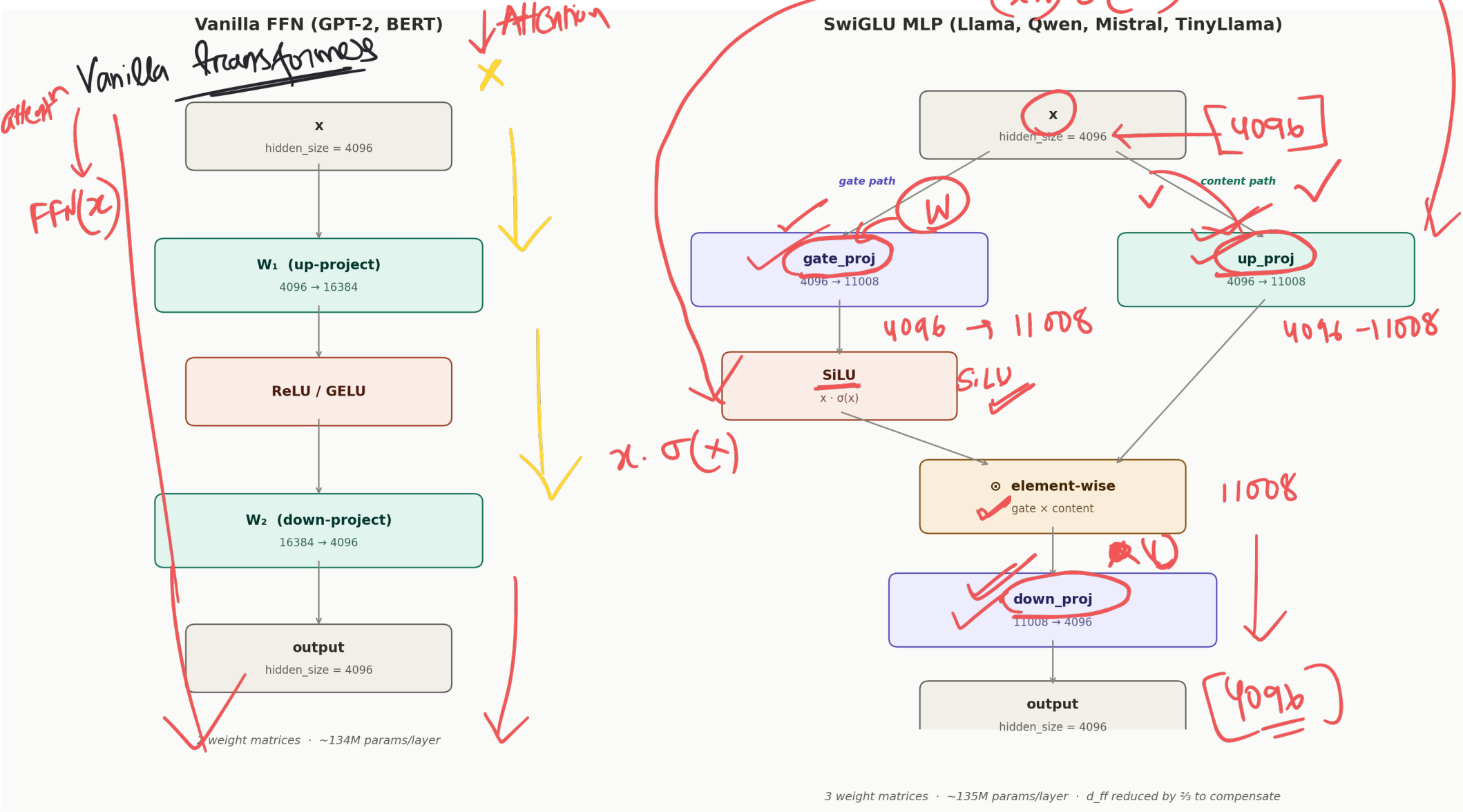


o proj



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one




Vanilla FFN (GPT-2, BERT)
x
hidden_size = 4096
W₁ (up-project)
4096 → 16384
ReLU / GELU
W₂ (down-project)
16384 → 4096
output
hidden_size = 4096
2 weight matrices · ~134M params/layer
SwiGLU MLP (Llama, Qwen, Mistral, TinyLlama)
x
hidden_size = 4096
gate path
content path
gate_proj
4096 → 11008
up_proj
4096 → 11008
SiLU
x · σ(x)
⊙ element-wise
gate × content
down_proj
11008 → 4096
output
hidden_size = 4096
3 weight matrices · ~135M params/layer · d_ff reduced by ⅔ to compensate

$FFN_{GLU}(x)$

↓ Attention layers

$(x \cdot W + b1)$ $\times$ $[1 \times 4096]$ σ $(x \cdot V + b2)$

W 11008 ↓ ✓ 11008

$[\quad]_{1 \times 11008}$ $[\quad]_{1 \times 11008}$

FFN(x)

↓ Attention

$x[1 \times 4096]$

↓ W1

11008

Gated Linear Unit

The original GLU formula captures this two-pathway structure:

$$\text{GLU}(x) = (xW + b) \otimes \sigma(xV + c)$$

Let's unpack each component to understand what it contributes:

The Value Pathway: $(xW + b)$

The first term computes a linear transformation of the input. This is the "content" that we might want to pass through to the next layer. Without the gate, this would just be a standard linear layer:

- $x \in \mathbb{R}^{d_{\text{model}}}$: the input vector (a token's representation)
- $W \in \mathbb{R}^{d_{\text{model}} \times d}$: the "value" projection weights that transform the input
- $b \in \mathbb{R}^{d}$: the value projection bias

The Gate Pathway: $\sigma(xV + c)$

The second term computes *another* linear transformation of the same input, but passes it through the sigmoid function to produce gate values:

features

(attention)

$(x \cdot W + b) \otimes \sigma(xV + c)$

$(x \cdot W + b)$

previous layers

9:20

↓ Attention layer

ReLU $(X \cdot W_1 + b_1) \cdot W_2 + b_2$

↑ Dim 1 ↑ Dim 2

Recalling the Standard FFN

The standard FFN applies a simple pattern: expand the dimension, apply nonlinearity, contract back:

$$\text{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2$$

where x is the input, W_1 and W_2 are weight matrices, b_1 and b_2 are bias vectors, and σ is a nonlinear activation function (e.g., ReLU or GELU).

The key insight is that the nonlinearity σ is applied *after* the expansion. This creates a high-dimensional feature space where the nonlinearity can carve out complex decision boundaries, before projecting back to the original dimension.

FFN(x)

Vanilla transformer

$= \text{ReLU}(x \cdot W_1 + b_1) W_2 + b_2$

x?

FFN(x) = ReLU $(x \cdot W_1 + b_1) W_2 + b_2$

attention layer

FFN(x)

↓ Attention Layer

x [1x4096]

↓ W1 Relu(x·W1+b1)

z = [ ] 1x11008

↓ W2 z·W2+b2

[ ] 1x4096

FFN_GLU(x)

↓ Attention layer

x [1x4096]

↓ W 4096x11008 (x·W+b) → [ ] 1x11008

↓ V 4096x11008 *learnt AF σ(x·V+c) → [ ] 1x11008

[ ] 1x11008

↓ W2 11008x4096

[ ] 1x4096

Attention layer

$$[(x \cdot W + b) \otimes \sigma(x \cdot V + c)] \cdot W2 + b2$$

Replacing Activation with Gating

GLU replaces the simple activation function with something more sophisticated: a learned gating mechanism. Instead of uniformly transforming all features with the same nonlinearity, GLU lets the network selectively amplify or suppress features based on the input itself:

$$\text{FFN}_{\text{GLU}}(x) = [(xW + b) \otimes \sigma(xV + c)]\, W_2 + b_2$$

Let's trace through what each component does in this context:

1. **Expansion to Hidden Space**: Both xW and xV project the input from d_{model} to the larger hidden dimension d_{ff}
2. **Gated Combination**: The sigmoid-gated multiplication produces a d_{ff}-dimensional hidden representation
3. **Contraction Back**: W_2 projects back to d_{model} for the residual connection

left

right

FFN_{GLU}

4096 → 11008

attention layers

$$FFN_{GLU}(x) = [(x \cdot W + b) \otimes \sigma(x \cdot V + c)] \cdot W2 + b2$$

Linear transformation of features from attention layer

The sigmoid function produces values between 0 and 1, making it a natural choice for gating:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where z is the input value (applied element-wise when the input is a vector).

The sigmoid's behavior at extreme values is what makes it effective as a gate:

- As $z \to -\infty$, $\sigma(z) \to 0$ (gate closes completely)
- As $z \to +\infty$, $\sigma(z) \to 1$ (gate opens fully)
- At $z = 0$, $\sigma(z) = 0.5$ (gate is half-open)

From Sigmoid to Swish: A Self-Gating Activation

Swish (also called SiLU for Sigmoid Linear Unit) offers an elegant solution. Instead of using sigmoid as a separate gate, Swish *embeds* gating directly into the activation function:

$$\text{Swish}(z) = z \cdot \sigma(z)$$

where z is the input value and $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function.

This formula is deceptively simple but powerful. Let's understand what it does:

- For large positive z: $\sigma(z) \approx 1$, so $\text{Swish}(z) \approx z$ (passes through like a linear function)
- For large negative z: $\sigma(z) \approx 0$, so $\text{Swish}(z) \approx 0$ (suppressed like ReLU)
- For small z near zero: both z and $\sigma(z)$ contribute, creating a smooth transition

10 · σ(10)

= 10 · 1 = 10

−10 · σ(−10)

= −10 * 0.0001

≈ 0

input → z

Sigmoid − σ(z)

ReLU − cReLU(z)

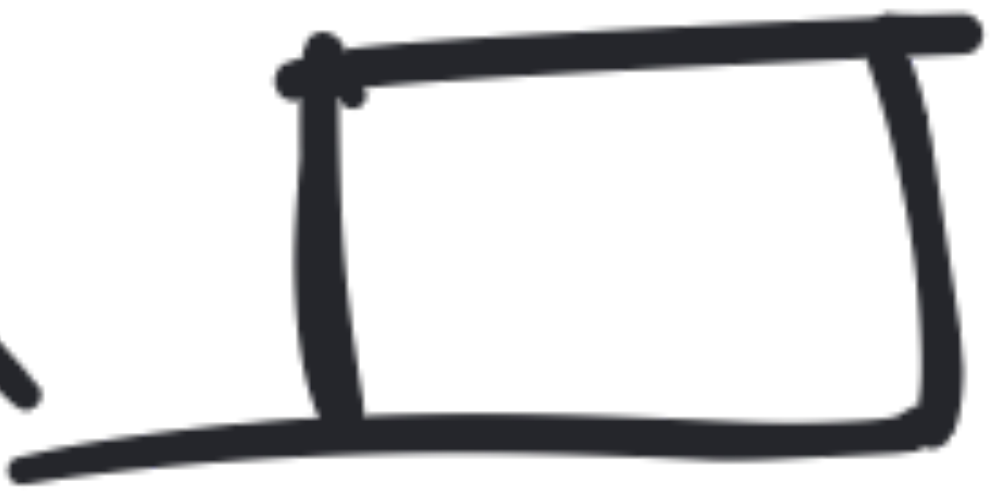
SiLU → z · σ(z)

ReLU(z) ✓

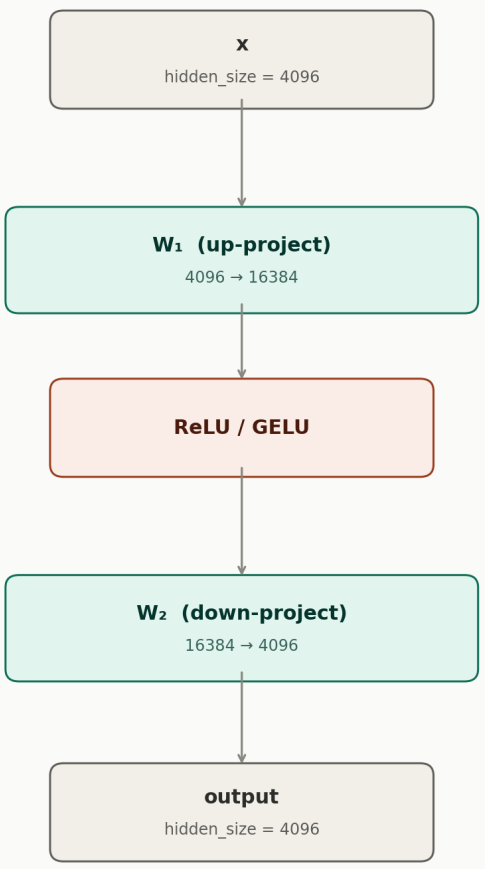
Swish(z) = x · σ(x)

GLU

Swish

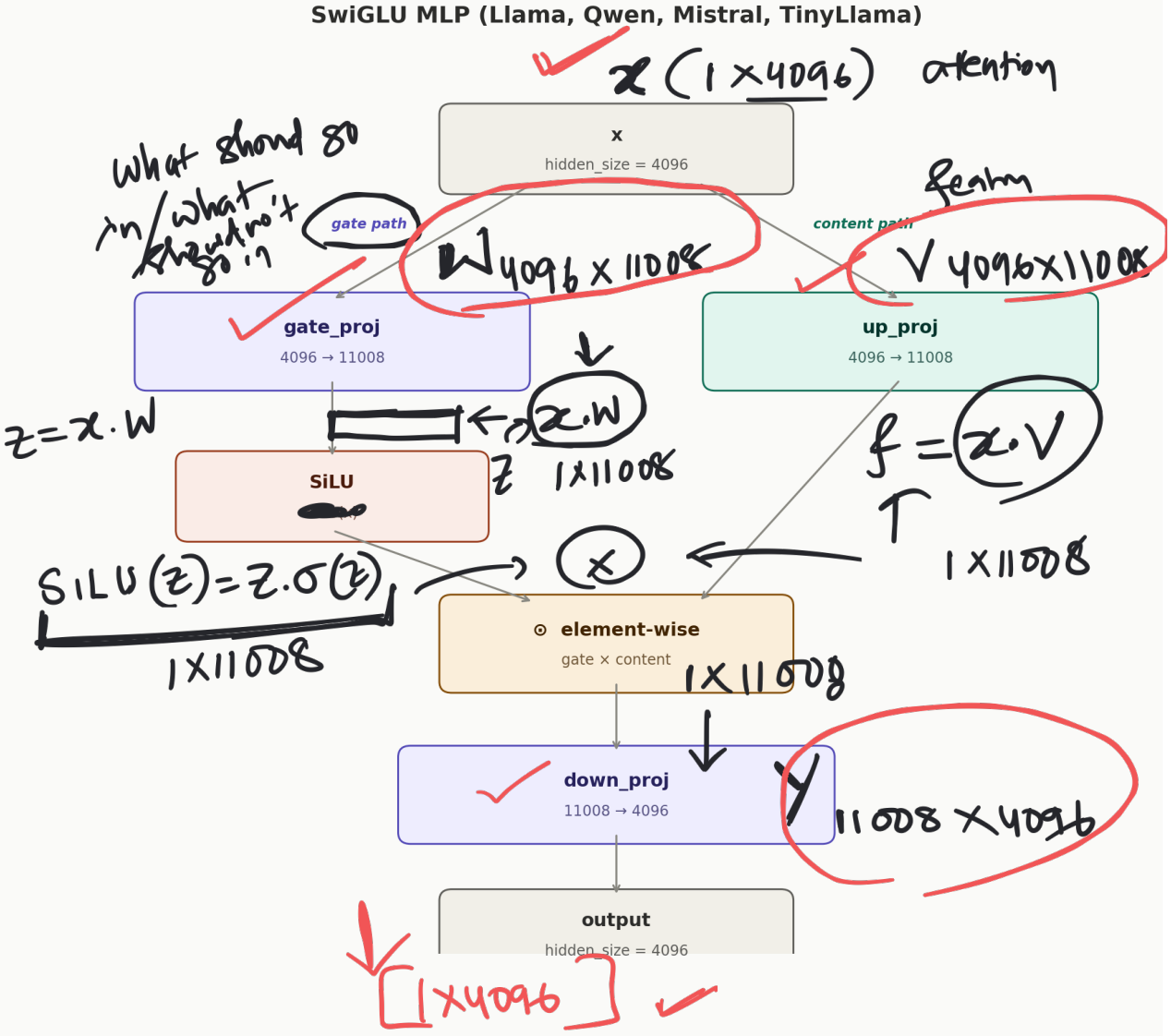


Vanilla FFN (GPT-2, BERT)



2 weight matrices · ~134M params/layer

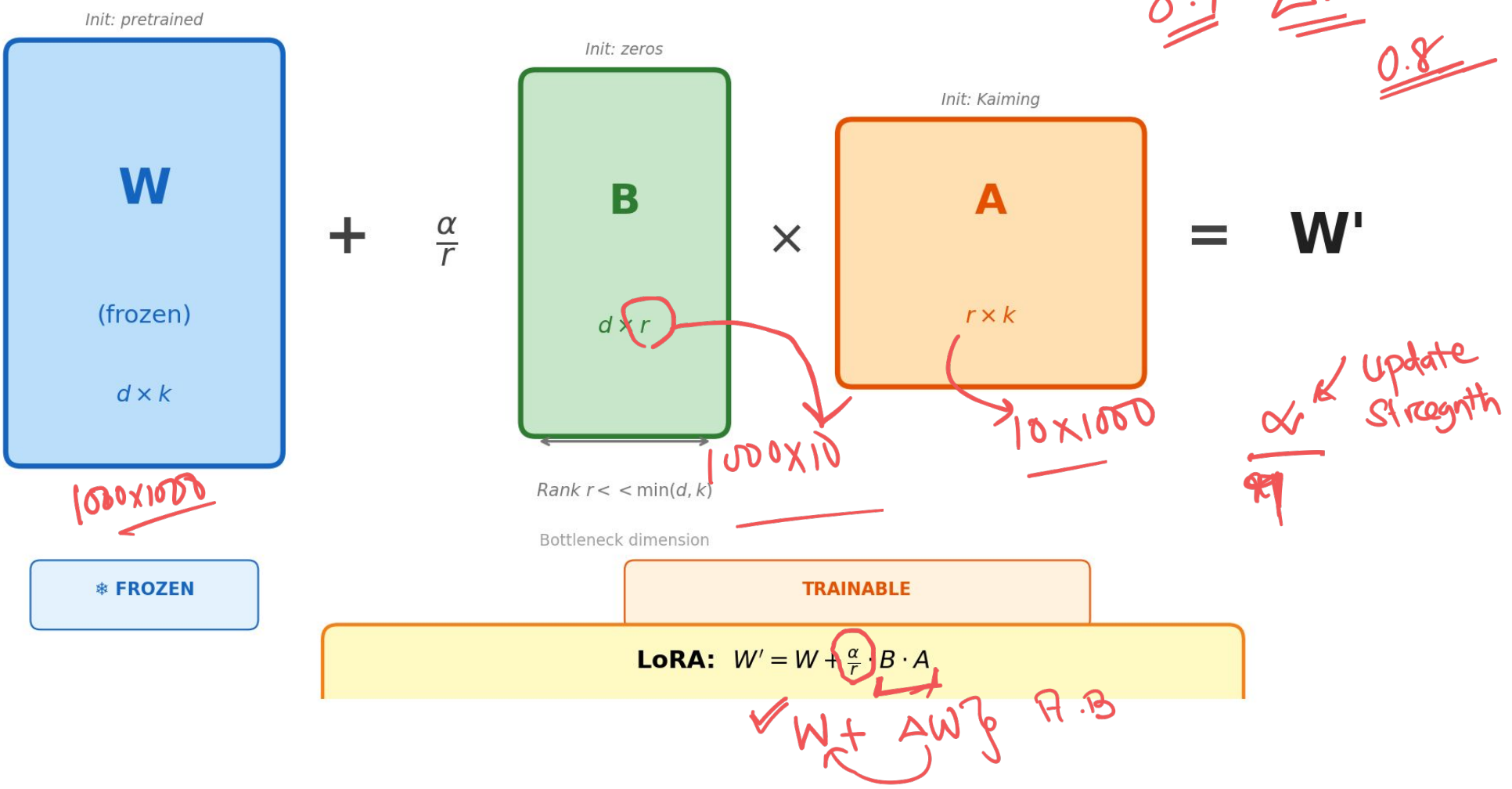
SwiGLU MLP (Llama, Qwen, Mistral, TinyLlama)

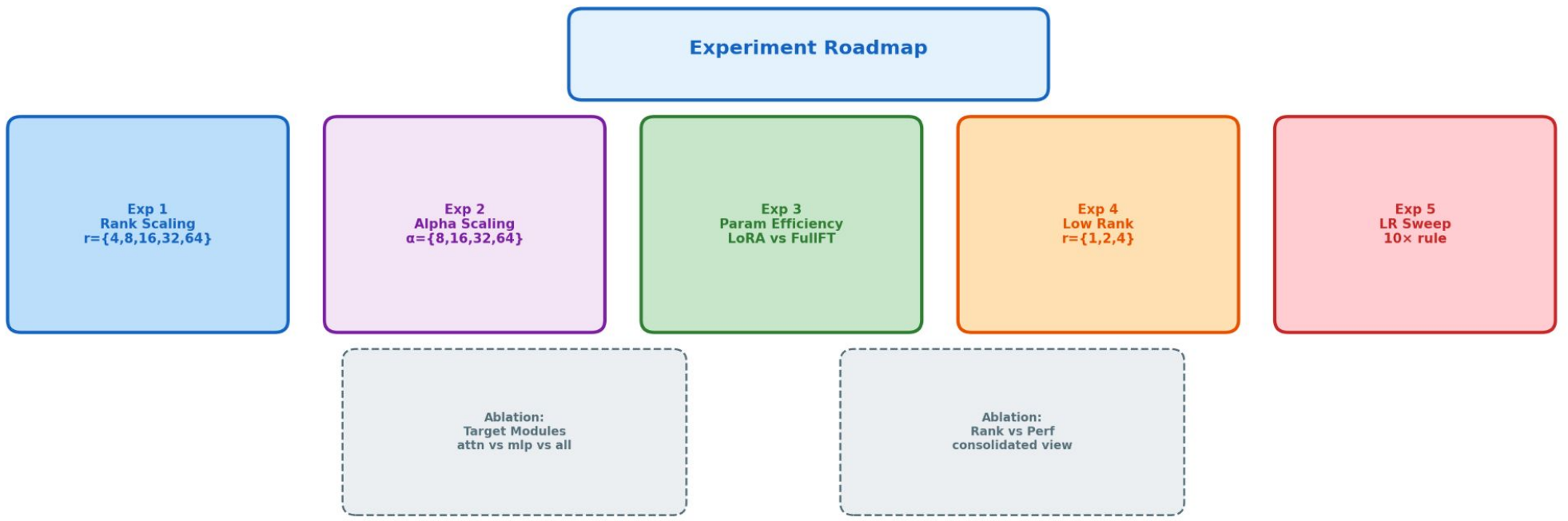


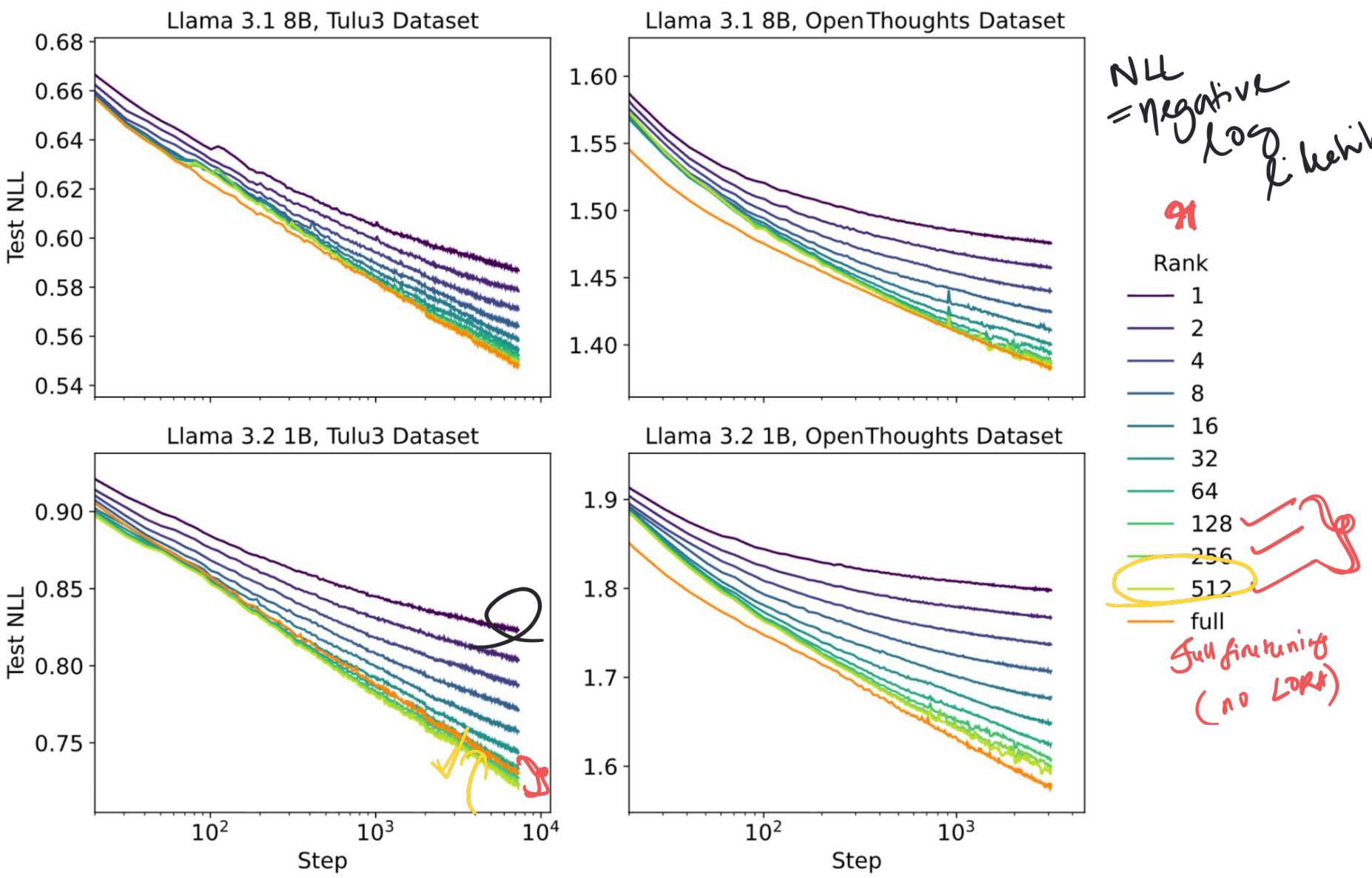
3 weight matrices · ~135M params/layer · d_ff reduced by ⅔ to compensate

LORA-2

LoRA Architecture: Low-Rank Adaptation




Experiment Roadmap
Exp 1
Rank Scaling
r={4,8,16,32,64}
Exp 2
Alpha Scaling
α={8,16,32,64}
Exp 3
Param Efficiency
LoRA vs FullFT
Exp 4
Low Rank
r={1,2,4}
Exp 5
LR Sweep
10× rule
Ablation:
Target Modules
attn vs mlp vs all
Ablation:
Rank vs Perf
consolidated view


Llama 3.1 8B, Tulu3 Dataset
Llama 3.1 8B, OpenThoughts Dataset
Test NLL
0.68
0.66
0.64
0.62
0.60
0.58
0.56
0.54
1.60
1.55
1.50
1.45
1.40
Llama 3.2 1B, Tulu3 Dataset
Llama 3.2 1B, OpenThoughts Dataset
0.90
0.85
0.80
0.75
1.9
1.8
1.7
1.6
10^2
10^3
10^4
Step
Rank
1
2
4
8
16
32
64
128
256
512
full
NLL = negative Log Likelihood
91
Full finetuning (no LoRA)

$W_{updated} = W - \partial W$

$W_{update} = W + \Delta W$

A, B → ∂A ∂B

rank = 10

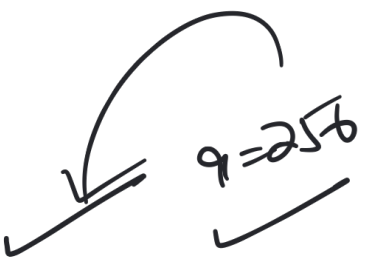
LORA

$(1000 \times 10) + (10 \times 1000)$

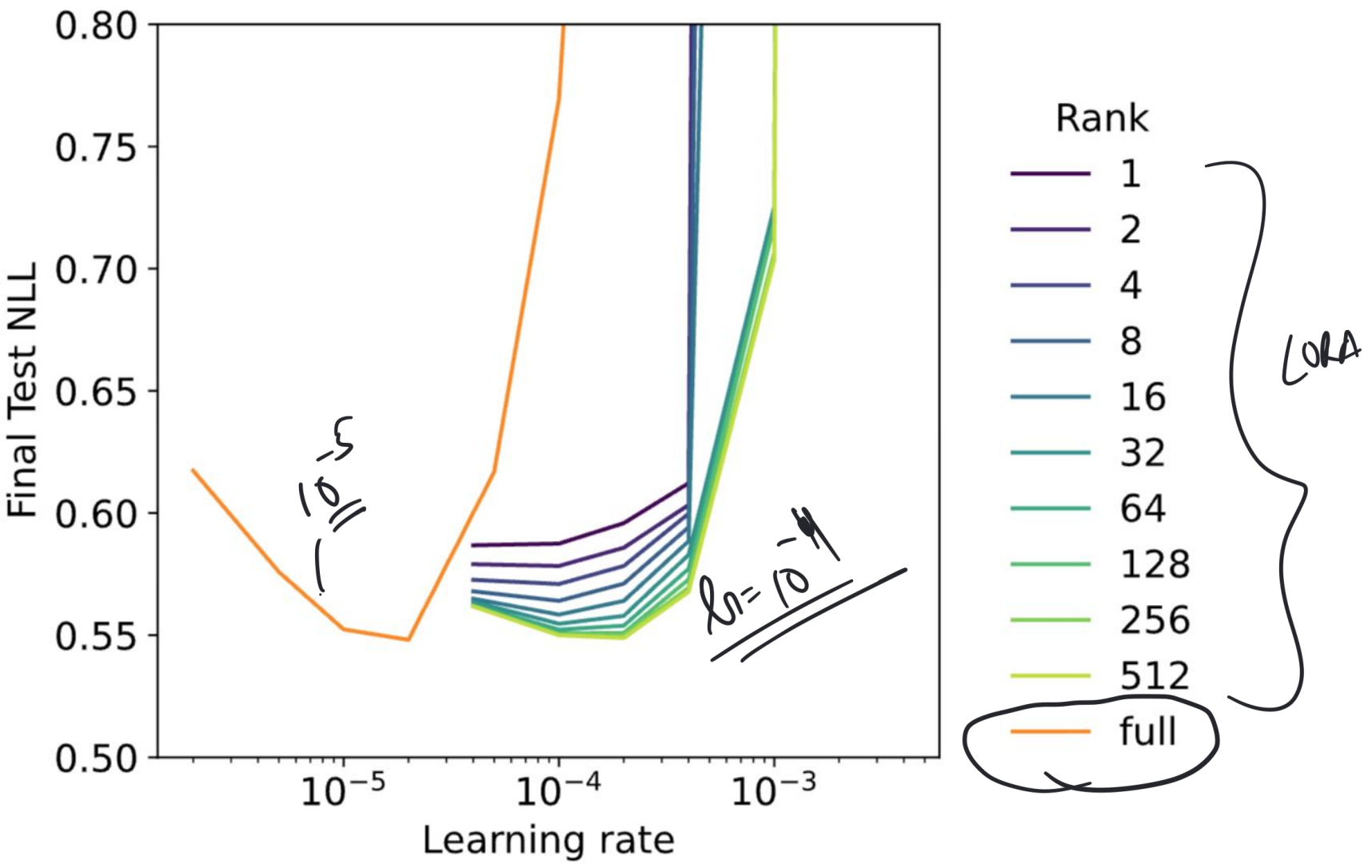
1000 X 1000

Full finetuning

1000 X 1000

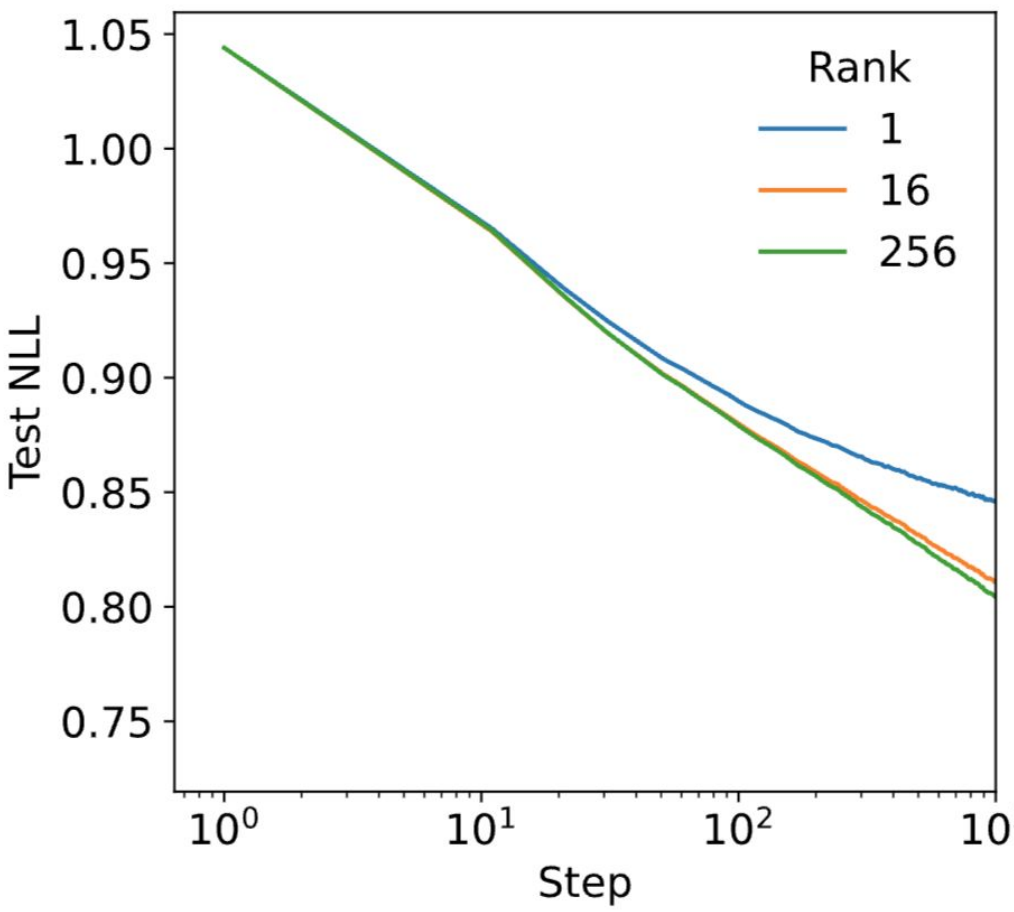


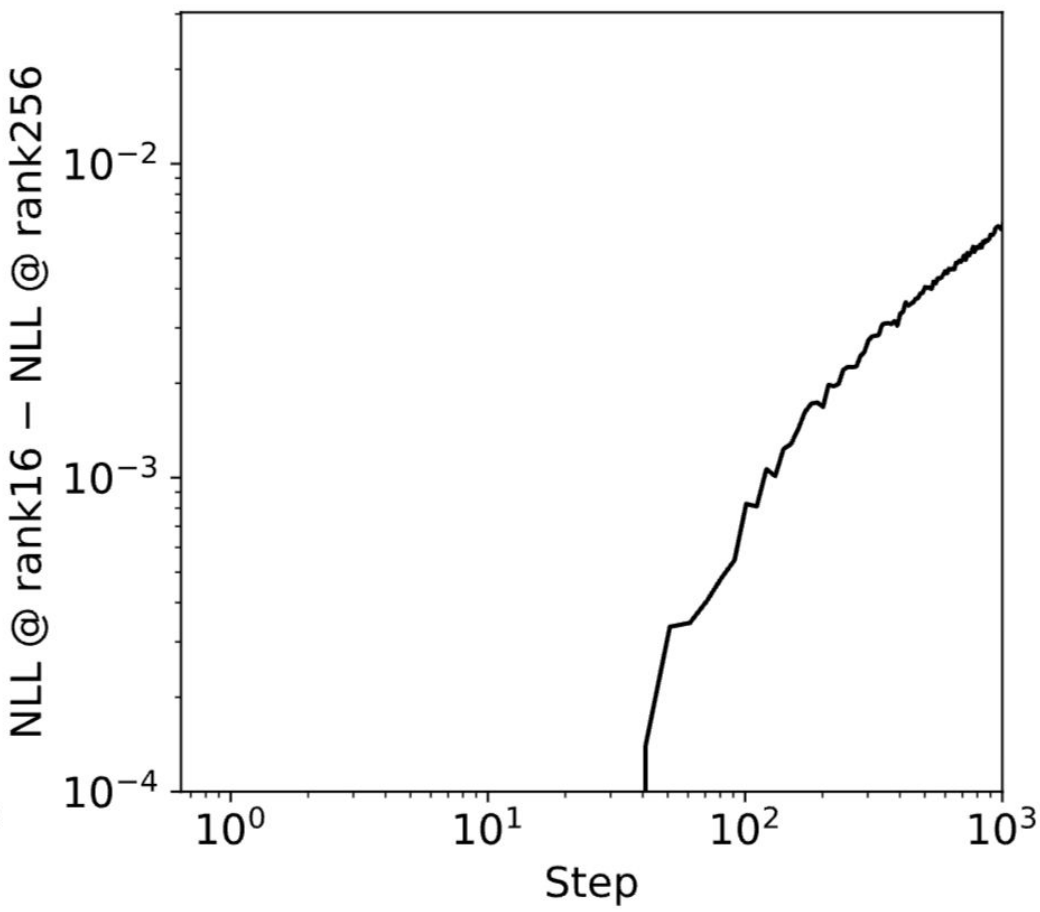
LoRA training curves for various ranks on Tulu3 and OpenThoughts3 datasets. FullFT and high-rank LoRAs have similar learning curves with loss decreasing linearly with the logarithm of steps. Lower-rank LoRAs fall off the minimum-loss curve when the adapter runs out of capacity. In the bottom plots (1B model) high-rank LoRA performs better than FullFT on one dataset and worse on the other. There might be some random variation in how LoRA performs on different datasets, due to differences in training dynamics or generalization behavior.

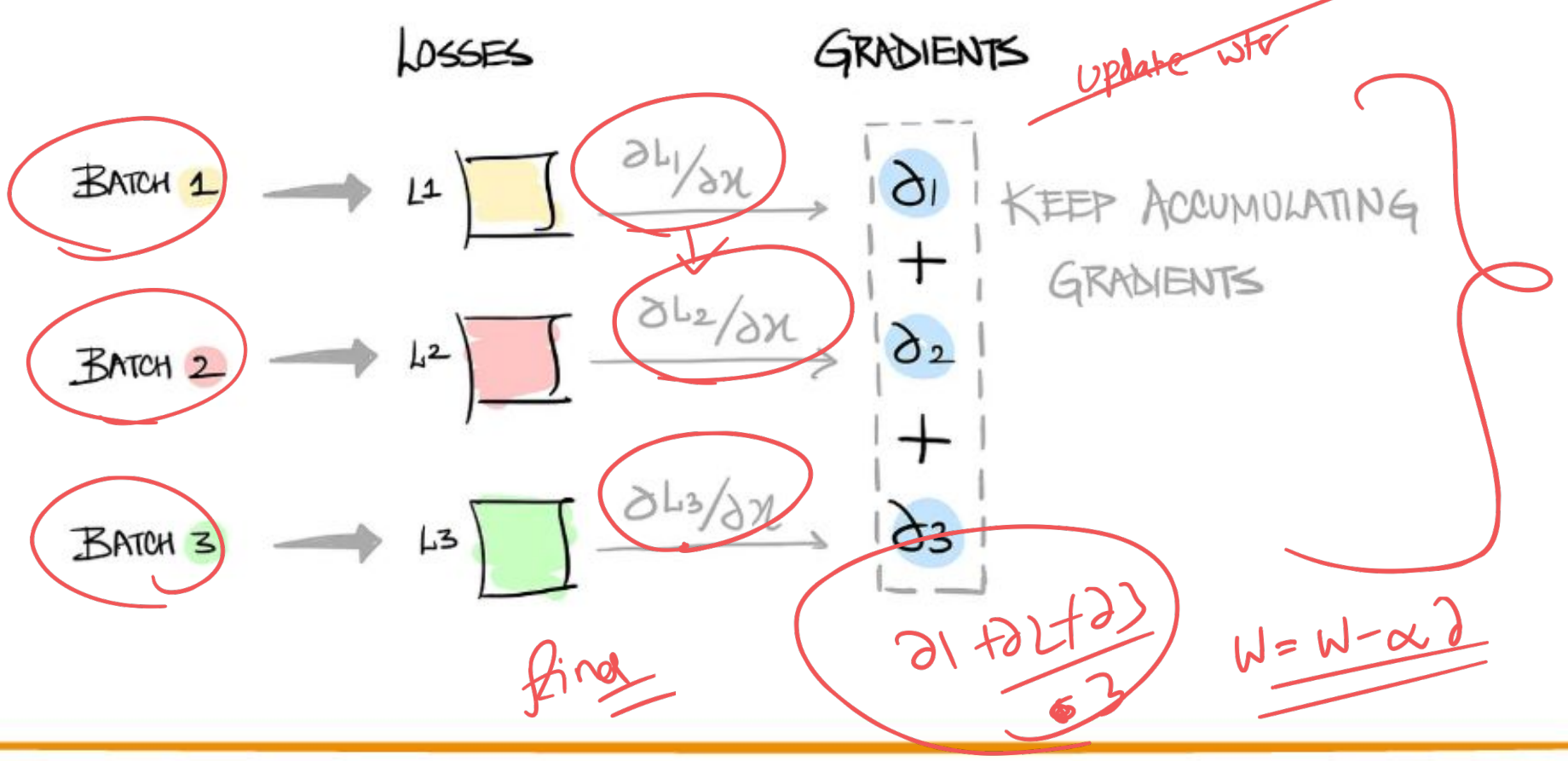

0.80
0.75
0.70
0.65
0.60
0.55
0.50
Final Test NLL
10^{-5}
10^{-4}
10^{-3}
Learning rate
Rank
1
2
4
8
16
32
64
128
256
512
full
LORA

Learning rate versus final loss for various LoRA ranks on Tulu3. Minimum loss is approximately the same for high rank LoRA and FullFT. Optimal LR is 10 times higher for LoRA.

Llama-3.1-1B, Tulu3 Dataset, same LR (1e-4), different ranks






LOSSES
GRADIENTS
Update wts
BATCH 1
L1
∂L1/∂x
∂1
+
KEEP ACCUMULATING GRADIENTS
BATCH 2
L2
∂L2/∂x
∂2
+
BATCH 3
L3
∂L3/∂x
∂3
find
∂1+∂2+∂3 / 3
W = W - α∂

Batch size = 10

Effective Batch size = 10 x 3
= 30

B1 = 10
B2 = 10
B3 = 10
} 3

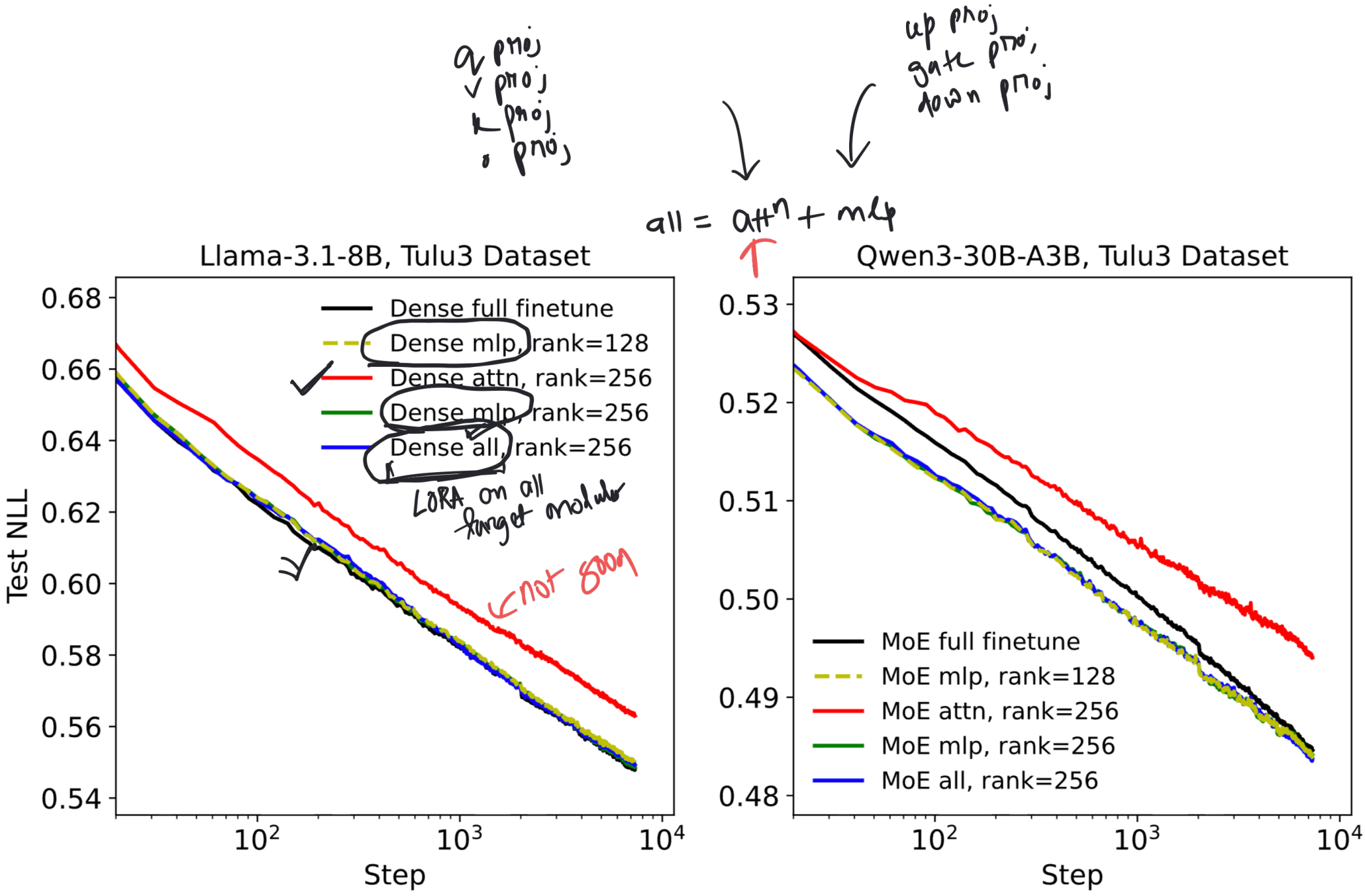
(30) → Gradients are stable

Without grad accum

```
# Training loop
for epoch in range(num_epochs):
    for i, data in enumerate(train_loader):
        inputs, labels = data

        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # Backward pass
        loss.backward()

        # Perform weight update
        optimizer.step()

        # Zero out gradients
        optimizer.zero_grad()
```

With grad accum

```
# Training loop
for epoch in range(num_epochs):
    for i, data in enumerate(train_loader):
        inputs, labels = data

        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # Backward pass and gradient accumulation
        loss.backward()

        # Check if it's time for a weight update
        if (i + 1) % accumulation_steps == 0:
            # Divide accumulated gradients by accumulation steps
            for param in model.parameters():
                param.grad /= accumulation_steps

            # Perform weight update
            optimizer.step()

            # Zero out gradients
            for param in model.parameters():
                param.grad.zero_()
```


q proj
v proj
k proj
o proj
up proj
gate proj
down proj
all = attn + mlp
Llama-3.1-8B, Tulu3 Dataset
Qwen3-30B-A3B, Tulu3 Dataset
Test NLL
Step
Dense full finetune
Dense mlp, rank=128
Dense attn, rank=256
Dense mlp, rank=256
Dense all, rank=256
LoRA on all target modules
not good
MoE full finetune
MoE mlp, rank=128
MoE attn, rank=256
MoE mlp, rank=256
MoE all, rank=256

Attention-only LoRA significantly underperforms MLP-only LoRA, and does not further improve performance on top of LoRA-on-MLP. This effect holds for a dense model (Llama-3.1-8B) and a sparse MoE (Qwen3-30B-A3B-Base).

(W) x FP = x·W

↑ Gradients

peft config

frozen

FP = x·(A·B) + x·W